



KPLABS Course

HashiCorp Certified: Terraform Associate

Domain 6

ISSUED BY

Zeal

REPRESENTATIVE

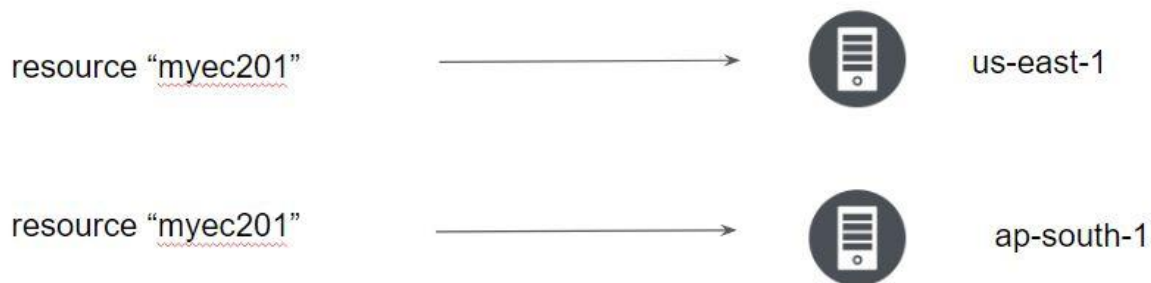
instructors@kplabs.in

Domain 6 - Security Primer

Module 1: Provider Configuration

Till now, we have been hardcoding the `aws-region` parameter within the `providers.tf` file.

This means that resources would be created in the region specified in the `providers.tf` file.



By default, resources use a default provider configuration inferred from the first word of the resource type name.

For example, a resource of type `aws_instance` uses the default (un-aliased) `aws` provider configuration unless otherwise stated.

To select an aliased provider for a resource or data source, set its provider meta-argument to a `<PROVIDER NAME>.<ALIAS>` reference:

```
resource "aws_instance" "foo" {  
  provider = aws.west  
  
  # ...  
}
```

Module 2: Handling Multiple AWS Profiles in Terraform

You can optionally define multiple configurations for the same provider, and select which one to use on a per-resource or per-module basis.

The primary reason for this is to support multiple regions for a cloud platform.

To include multiple configurations for a given provider, include multiple provider blocks with the same provider name, but set the alias meta-argument to an alias name to use for each additional configuration. For example:

```
# The default provider configuration
provider "aws" {
    region = "us-east-1"
}

# Additional provider configuration for west coast region
provider "aws" {
    alias   = "west"
    region = "us-west-2"
}
```

The provider block without alias set is known as the default provider configuration.

When an alias is set, it creates an additional provider configuration.

For providers that have no required configuration arguments, the implied empty configuration is considered to be the default provider configuration.

Module 3: Sensitive Parameter

With the organization managing its entire infrastructure in terraform, it is likely that you will see some sensitive information embedded in the code.

When working with a field that contains information likely to be considered sensitive, it is best to set the Sensitive property on its schema to true

```
output "db_password" {
    value      = aws_db_instance.db.password
    description = "The password for logging in to the database."
    sensitive  = true
}
```

Setting the sensitive to “true” will prevent the field's values from showing up in CLI output and in Terraform Cloud

It will not encrypt or obscure the value in the state, however.

```
C:\Users\Zeal Vora\Desktop\terraform\sensitive data>terraform apply
```

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
db_password = <sensitive>
```